claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a2e80e7614c44fdc3.jsonl`
- SHA-256: `66954d8c912aec28a5f636b90e6faee9af644abc6a7feabb4a5b2dc102aeaee2`
- Source modified: `2026-07-06T17:13:19+00:00`
- Imported at: `2026-07-08T15:59:36+00:00`
- project: `subagents`
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`

Transcript

1. user (2026-07-06T17:10:39.648Z)

Read-only investigation in the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer. I need to understand exactly what determines WASB shuttle-detector inference wall-clock time, so I can prove that downsampling a video would make it process faster.

Start with backend/pipeline/detectors/native_wasb_runner.py and the WASB inference entrypoint it shells out to (search for wasb_infer.py, "infer", the command it builds around repo_dir "~/models/WASB-SBDT"; the script may live in the repo under backend/ or tools/, or be expected inside the cloned WASB-SBDT repo). Also read backend/pipeline/segmenters/trajectory_hybrid.py.

Answer these CONCRETELY with file:line references and actual values:
1. Per-frame work: does the WASB network resize every input frame to a FIXED network input resolution (what resolution ? e.g. 512x288, 288x512, WxH)? Does SOURCE resolution (1920x1080 vs 1280x720) affect the forward-pass cost per frame, or only decode/resize cost?
2. Frame selection: are ALL frames processed, or is there an fps/stride subsampling knob? If a 60fps video is fed, does it process 60 frames per second of video or fewer? Where is that controlled?
3. Throughput levers: what exactly do batch_size and prefetch_batches do (native_wasb_runner.py NativeWasbConfig)? How would raising them change GPU throughput? What are the CURRENT serving defaults (config.json indexing.wasb)?
4. The "tuned proxy resolution" phrase in the trajectory_hybrid warning ("input resolution differs from the tuned proxy resolution") ? where is that resolution defined and what is its value?
5. Is there ANY existing spatial-downscale, fps-reduction, or proxy-generation step already in the detection code path? If so, where and is it enabled by default?

Be precise and quote the code. Do not modify anything. Return a concise findings report.

2. assistant (2026-07-06T17:10:41.386Z)

I'll systematically explore the codebase to answer these questions about WASB inference performance characteristics.

3. user (2026-07-06T17:10:41.843Z)

backend\pipeline\detectors\native_wasb_runner.py

4. user (2026-07-06T17:10:41.891Z)

backend\pipeline\detectors\wasb_infer.py

5. user (2026-07-06T17:10:41.976Z)

backend\pipeline\segmenters\trajectory_hybrid.py

6. user (2026-07-06T17:10:42.020Z)

```
config.json
artifacts\cloudrun_lightup\config.json
```

7. assistant (2026-07-06T17:10:43.412Z)

Now let me read the key files:

8. user (2026-07-06T17:10:44.117Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8       see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34      not the numbers.
35      """
36
37      import logging
38      import os
39      import shutil
40      import subprocess
```

```
    from dataclasses import dataclass
    from typing import List, Optional, Tuple

    from backend.config.models import IndexingConfig
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext

    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    logger = logging.getLogger(__name__)

    # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
    # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
    _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")

    # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
    # so the "cuda True/False" string they parse can never drift apart.
    _CUDA_PROBE_CODE = (
        "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
    )


    @dataclass
    class NativeWasbConfig:
        """Resolved settings for a Linux-native WASB run.

        Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
        over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
        defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
        """

        repo_dir: str = "~/models/WASB-SBDT"  # native clone of nttcom/WASB-SBDT
        python_bin: str = (
            "python"  # the `wasb` conda env python (invoked by path, no activate)
        )
        weights_path: str = ""  # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
        sport: str = "badminton"
        device: str = "cuda"  # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
        scratch_dir: str = ""  # frame-extraction scratch (env); "" = next to the output dir
        timeout_sec: int = 1800  # 30 min; a hung GPU call is killed (0 = no timeout)
        keep_frames: bool = (
            False  # retain the decoded frame cache after success (debug only)
        )
        # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
        # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
        # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
        stream_video: bool = True
        # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
        # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
        batch_size: int = 0
        prefetch_batches: int = 0

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
            if isinstance(idx_cfg, dict):
                idx_cfg = IndexingConfig(**idx_cfg)
```

```
            w = idx_cfg.wasb
            env = os.environ.get
            # stream_video is tri-state in config: None/absent => native default (stream);
an
            # explicit True/False still wins (e.g. False for a container cv2 can't seek).
            sv = w.stream_video
            return cls(
                repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
                python_bin=env("WASB_PYTHON") or w.python_bin,
                weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
                sport=w.sport,
                device=env("WASB_DEVICE") or w.device,
                scratch_dir=env("WASB_SCRATCH")
                or env("RALLY_SCRATCH")
                or env("TMPDIR")
                or "",
                timeout_sec=w.timeout_sec,
                keep_frames=w.keep_frames,
                stream_video=(True if sv is None else bool(sv)),
                batch_size=int(w.batch_size or 0),
                prefetch_batches=int(w.prefetch_batches or 0),
            )


    class NativeWasbRunner(DetectorRunner):
        """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""

        def __init__(self, cfg: NativeWasbConfig):
            self.cfg = cfg
            # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
            # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
            self._cuda_available: Optional[bool] = None

        # --- low-level native exec (the single place tests patch)
-------------------------
        def _run(
            self, argv: List[str], cwd: Optional[str] = None
        ) -> subprocess.CompletedProcess:
            logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
            return subprocess.run(
                argv,
                cwd=cwd,
                capture_output=True,
                text=True,
                timeout=(self.cfg.timeout_sec or None),
            )

        # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
        def _probe_cuda(self) -> bool:
            """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
            (missing python / timeout / import error) is treated as 'no CUDA'."""
            try:
                res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
            except (FileNotFoundError, subprocess.TimeoutExpired):
                return False
            return res.returncode == 0 and "cuda True" in (res.stdout or "")

        def _cuda_is_available(self) -> bool:
            """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
```

```
155             if self._cuda_available is None:
156                 self._cuda_available = self._probe_cuda()
157             return self._cuda_available
158 
159         def _effective_device(self) -> str:
160             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162             if self.cfg.device == AUTO:
163                 return DeviceContext(AUTO, self._cuda_is_available()).effective
164             return self.cfg.device
165 
166         def _repo_src(self) -> str:
167             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168 
169         def _copy_infer_script(self) -> bool:
170             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171             repo_src = self._repo_src()
172             try:
173                 os.makedirs(repo_src, exist_ok=True)
174                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175                 return True
176             except OSError as e:
177                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178                 return False
179 
180         def _rm_rf(self, path: Optional[str]) -> None:
181             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182             if path:
183                 shutil.rmtree(path, ignore_errors=True)
184 
185         def healthcheck(self) -> Tuple[bool, str]:
186             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187             if not self.cfg.weights_path:
188                 return False, (
189                     "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
190                 )
191             weights = os.path.expanduser(self.cfg.weights_path)
192             if not os.path.exists(weights):
193                 return False, f"WASB weights not found at {weights}"
194             repo_src = self._repo_src()
195             if not os.path.isdir(repo_src):
196                 return False, (
197                     f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198                     f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
199                 )
200             # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201             # argparse rc=2 from wasb_infer.py at run time.
202             if self.cfg.device not in VALID_DEVICES:
203                 return False, (
204                     f"unknown device {self.cfg.device!r} ? valid: {',
'.join(VALID_DEVICES)}."
205                 )
206             try:
207                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
208             except FileNotFoundError:
209                 return (
210                     False,
211                     f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
```

```
                )
            except subprocess.TimeoutExpired:
                return False, "native torch healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return (
                    False,
                    f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
                )
            # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
            self._cuda_available = "cuda True" in out
            ctx = DeviceContext(self.cfg.device, self._cuda_available)
            if ctx.cuda_required_but_missing:
                # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
                # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
                return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
            if ctx.fell_back:
                logger.warning(
                    "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
                    "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
                    out,
                )
            return True, out

        def run_predict(
            self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
        ) -> Optional[str]:
            """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.

            Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
            runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
            same downstream contract, so the parity comparison is apples-to-apples.
            """
            output_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_dir, exist_ok=True)
            # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
            norm = str(video_win_path).replace("\\", "/")
            stem, _ext = os.path.splitext(os.path.basename(norm))
            # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
            key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
            out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

            if not self._copy_infer_script():
                logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
                return None

            weights = os.path.expanduser(self.cfg.weights_path)
            argv = [
                self.cfg.python_bin,
                "wasb_infer.py",
                "--video",
                os.path.abspath(str(video_win_path)),
            ]
            frames_dir: Optional[str] = None
            if self.cfg.stream_video:
                # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
```

```
                argv.append("--stream-video")
            else:
                scratch = self.cfg.scratch_dir or output_dir
                frames_dir = os.path.join(scratch, f"{key}_frames")
                argv += ["--frames_out_dir", frames_dir]
            # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
            # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
            device = self._effective_device()
            argv += [
                "--weights",
                weights,
                "--sport",
                self.cfg.sport,
                "--device",
                device,
                "--out",
                out_csv,
            ]
            # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
            # byte-parity guarantee) is unchanged when the knobs are unset.
            if self.cfg.batch_size:
                argv += ["--batch-size", str(self.cfg.batch_size)]
            if self.cfg.prefetch_batches:
                argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]

            logger.info(
                "Running native WASB inference (device=%s%s) on %s ...",
                device,
                " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
                os.path.basename(norm),
            )
            try:
                res = self._run(argv, cwd=self._repo_src())
            except FileNotFoundError:
                logger.error(
                    "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
                )
                return None
            except subprocess.TimeoutExpired:
                logger.error(
                    "WASB native inference timed out after %ss.", self.cfg.timeout_sec
                )
                return None
            if res.returncode != 0:
                logger.error(
                    "WASB native inference failed:\n%s",
                    (res.stderr or res.stdout or "")[-2000:],
                )
                return None
            if not os.path.exists(out_csv):
                logger.error(
                    "WASB native finished but expected CSV not found at %s", out_csv
                )
                return None
            logger.info("WASB trajectory CSV: %s", out_csv)

            # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
            # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
            # (Streaming never materializes frames, so there is nothing to drop on that
path.)
```

```
327             if not self.cfg.keep_frames and frames_dir:
328                 logger.info(
329                     "Cache hygiene: removing frame cache for %s "
330                     "(set indexing.wasb.keep_frames=true to retain).",
331                     key,
332                 )
333                 self._rm_rf(frames_dir)
334             return out_csv
335 
336         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338         trajectory_to_action_windows = staticmethod(
339             TrackNetRunner.trajectory_to_action_windows
340         )
341 
```

9. user (2026-07-06T17:10:44.787Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2 
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6 
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9 
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17 
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20 
21      import concurrent.futures
22      import logging
23      import os
24      import time
25      from typing import Any, Dict, List, Optional, Tuple
26 
27      import cv2
28 
29      from backend.config.models import IndexingConfig
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.segmenters.base import BasePlaySegmenter
32      from backend.providers import provider_registry
33      from backend.sports import sport_registry
34      from backend.utils.paths import output_base
35      from backend.utils.video import extract_video_chunk, get_video_duration
36 
37      logger = logging.getLogger(__name__)
38 
39 
40      def _fmt_ts(seconds: float) -> str:
41          seconds = max(0, int(seconds))
42          h, rem = divmod(seconds, 3600)
43          m, s = divmod(rem, 60)
44          return f"{h:02d}:{m:02d}:{s:02d}"
45 
46 
```

```
    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
            raise NotImplementedError

        def _build_native_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
            native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
            raise NotImplementedError(
                f"detector_impl='native' is not supported for the "
                f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
                f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
            )

        def _resolve_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.

            ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
            ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
            is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
            in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
            ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
            (the WSL runner) so the default production path is unchanged
            (``docs/DECOUPLED_COMPUTE/``).
            """
            # Safety net: accept raw dicts from legacy / test callers.
            if isinstance(idx_cfg, dict):
                idx_cfg = IndexingConfig(**idx_cfg)
            impl = (
                os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
            ).lower()
            if impl == "stub":
                from backend.pipeline.detectors.stub_runner import (
                    StubConfig,
                    StubDetectorRunner,
                )

                logger.info(
                    "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                    self.display_label or "trajectory",
                )
```

```
                return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
                    "detector_impl": "stub"
                }
            if impl in ("native", "native_wasb", "wasb_native"):
                logger.info(
                    "%s: detector_impl=native ? Linux-native runner (no WSL).",
                    self.display_label or "trajectory",
                )
                return self._build_native_runner(idx_cfg)
            return self._build_runner(idx_cfg)

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(
            self,
            cw: Dict[str, Any],
            points: List[Any],
            fps: float,
            frame_width: float,
            video_path: str,
            idx_cfg: "IndexingConfig",
        ) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(
            self,
            windows: List[Dict[str, Any]],
            window_stats: List[Dict[str, Any]],
            idx_cfg: "IndexingConfig",
```

```
        ) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(  # type: ignore[override]
            self,
            video_id: str,
            video_path: str,
            player_pool: Optional[List[str]] = None,
            max_frames: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.sport
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.indexing
            skip_ai = bool(idx_cfg.skip_ai_handoff)
            provider_name = self.config.ai_provider
            if skip_ai:
                model_name = "local-noai"
                logger.info(
                    "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
                    "be emitted as rallies; no %s handoff, no API key needed.",
                    label,
                    provider_name,
                )
            else:
                model_name = self.config.ai_model
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint

                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(
                        provider_name, api_key=api_key, model_name=model_name
                    )
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)
```

```
227
228             # --- Runner config + windowing thresholds ---
229             # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
230             # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
231             # it delegates to the subclass _build_runner (the real WSL/native runner).
232             try:
233                 runner, runner_params = self._resolve_runner(idx_cfg)
234             except NotImplementedError as e:
235                 # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
236                 # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
237                 logger.error("%s: %s", label, e)
238                 return []
239
240             velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
241             merge_gap = idx_cfg.dead_time_merge_gap
242             min_window_duration = idx_cfg.min_action_window_duration
243             # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
244             max_window_duration = idx_cfg.max_window_duration
245             window_min_split_gap = idx_cfg.window_min_split_gap
246             window_hard_cap = idx_cfg.window_hard_cap
247             window_inactivity_end = idx_cfg.window_inactivity_end
248             inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
249             inactivity_min_density = idx_cfg.inactivity_min_density
250             inactivity_temporal_density = idx_cfg.inactivity_temporal_density
251             window_blob_fallback = idx_cfg.window_blob_fallback
252             window_blob_factor = idx_cfg.window_blob_factor
253             handoff_padding = idx_cfg.ai_handoff_padding
254             ai_max_workers = idx_cfg.ai_max_workers
255             log_candidates = idx_cfg.log_yolo_candidates
256             store_candidates = idx_cfg.store_yolo_candidates
257
258             detection_params = {
259                 "detector": self.name,
260                 "velocity_thresh": velocity_thresh,
261                 "merge_gap": merge_gap,
262                 "min_action_window_duration": min_window_duration,
263                 **runner_params,
264             }
265
266             # --- Phase 1: env healthcheck (fail fast with a clear message) ---
267             ok, msg = runner.healthcheck()
268             if not ok:
269                 logger.error("%s detector environment not ready: %s", label, msg)
270                 return []
271             logger.info("%s detector env OK: %s", label, msg)
272
273             # --- Phase 2: trajectory -> candidate rally windows ---
274             # Trajectory detectors process the whole video in one pass (they carry
275             # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
276             t_start = time.time()
277             out_dir = os.path.join(output_base(self.config), self.family)
278             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
279             if not csv_path:
280                 logger.error("%s produced no trajectory; aborting.", label)
281                 return []
282
283             points = runner.parse_trajectory_csv(csv_path)
284             logger.info(
285                 "Parsed %d trajectory points (%d visible).",
286                 len(points),
287                 sum(1 for p in points if p.visible),
288             )
```

```
289
290             all_candidate_windows = runner.trajectory_to_action_windows(
291                 points,
292                 fps=fps,
293                 frame_width=frame_width,
294                 chunk_start=0.0,
295                 velocity_thresh=velocity_thresh,
296                 merge_gap=merge_gap,
297                 min_window_duration=min_window_duration,
298                 chunk_index=0,
299                 max_window_duration=max_window_duration,
300                 min_split_gap=window_min_split_gap,
301                 window_hard_cap=window_hard_cap,
302                 window_inactivity_end=window_inactivity_end,
303                 inactivity_rally_thresh=inactivity_rally_thresh,
304                 inactivity_min_density=inactivity_min_density,
305                 inactivity_temporal_density=inactivity_temporal_density,
306                 window_blob_fallback=window_blob_fallback,
307                 window_blob_factor=window_blob_factor,
308             )
309             logger.info(
310                 "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
311                 label,
312                 time.time() - t_start,
313                 len(all_candidate_windows),
314             )
315
316             if log_candidates:
317                 for cw in all_candidate_windows:
318                     logger.info(
319                         f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
320                         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
321                         f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
322                     )
323
324             # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
325             # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
326             # persistence and the handoff gate below.
327             window_stats = [
328                 self._enrich_candidate_stats(
329                     cw, points, fps, frame_width, video_path, idx_cfg
330                 )
331                 for cw in all_candidate_windows
332             ]
333
334             # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
335             candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
336             if store_candidates:
337                 for i, cw in enumerate(all_candidate_windows):
338                     candidate_ids[i] = self.db.add_candidate_segment(
339                         video_id,
340                         detector=self.name,
341                         start_time=cw["start"],
342                         end_time=cw["end"],
343                         chunk_index=cw["chunk_index"],
344                         confidence=min(1.0, cw["peak_velocity"]),
345                         stats=window_stats[i],
346                         detection_params=detection_params,
347                     )
348
349             if not all_candidate_windows:
350                 return []
```

```
351
352             # --- Phase 3: AI provider handoff over padded candidate clips ---
353             # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
354             # candidates above stay the full superset ? only the served play_segments are
gated.
355             handoff_indices = self._select_for_handoff(
356                 all_candidate_windows, window_stats, idx_cfg
357             )
358
359             ai_segments: List[dict] = []
360             if skip_ai:
361                 # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
362                 # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
363                 # falls back to the duration-based estimate (no AI exchange count).
364                 ai_segments = [
365                     {
366                         "start_time": all_candidate_windows[wi]["start"],
367                         "end_time": all_candidate_windows[wi]["end"],
368                         "shuttle_exchange_count": None,
369                         "shot_count_confidence": "LocalNoAI",
370                         "_candidate_id": candidate_ids[wi],
371                     }
372                     for wi in handoff_indices
373                 ]
374                 logger.info(
375                     "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
376                     "(no AI handoff).",
377                     len(ai_segments),
378                 )
379             else:
380                 handoff_dir = os.path.join(
381                     output_base(self.config), f"chunks_{provider_name}_handoff"
382                 )
383                 os.makedirs(handoff_dir, exist_ok=True)
384                 logger.info(
385                     "Phase 3: %s validation on %d candidate windows...",
386                     provider_name,
387                     len(handoff_indices),
388                 )
389
390                 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
391                     w_start = max(0, window["start"] - handoff_padding)
392                     w_end = min(video_duration, window["end"] + handoff_padding)
393                     w_dur = w_end - w_start
394                     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
395                     if not extract_video_chunk(
396                         video_path, w_start, w_dur, w_path, reencode=True
397                     ):
398                         return []
399                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
400                     segments, metrics = provider.analyze_video(
401                         w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
402                     )
403                     # B2: a provider failure returns [] with metrics["error"] set ? that is
404                     # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
405                     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
406                     if metrics.get("error"):
407                         logger.warning(
408                             "[Handoff %d] provider error ? window %d skipped: %s",
```

```
                            wi + 1,
                            wi,
                            metrics["error"],
                        )
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning(
                                "Dropping segment with unparseable timestamps: %s ? %s",
                                seg,
                                e,
                            )
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(
                    max_workers=ai_max_workers
                ) as executor:
                    futures = [
                        executor.submit(
                            process_candidate_window, wi, all_candidate_windows[wi]
                        )
                        for wi in handoff_indices
                    ]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info(
                    "Phase 3 completed. AI confirmed %d semantic play segments.",
                    len(ai_segments),
                )

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
                    )
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append(
```

```
                            {
                                "id": segment_id,
                                "start_time": start,
                                "end_time": end,
                                "duration": dur,
                                "metadata": metadata,
                            }
                        )
                except Exception as e:
                    logger.warning("Skipping invalid segment struct: %s", e)

            return segments_data

```

10. user (2026-07-06T17:10:44.886Z)

```
      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).

      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
      own building blocks (ImageDataset transform + detector + online tracker) to run on
      any folder of extracted frames and emit a trajectory CSV in the exact shape the
      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:

          Frame,Visibility,X,Y

      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.

      Resumable across reboots
      ------------------------
      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
      the slow, expensive part; the CPU tracker pass that turns detections into a
      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
      resume mid-stream). We exploit that split:

        * The detector pass writes its per-window raw outputs incrementally to
          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
          most ``--flush-every`` windows of GPU work instead of the whole pass.
        * On restart we replay the cached windows, skip the DataLoader ahead to the first
          un-cached window, and continue. Once every window is cached, the detector pass
          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
          not the GPU hour.

      The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
      to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
      weights, sport, window size, or frame count change.

      Usage (in WSL, from ~/models/WASB-SBDT/src):
          python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
              --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
      """

      import argparse
      import csv
      import glob
      import json
      import logging
      import os
      import os.path as osp
      from collections import defaultdict
```

```
    from contextlib import contextmanager

    logger = logging.getLogger(__name__)

    CACHE_VERSION = 1
    _DET_FILE = "det_raw.jsonl"
    _MANIFEST_FILE = "manifest.json"


    def _build_cfg(weights: str, sport: str, device: str = "cuda"):
        """Compose the WASB eval config via Hydra, overriding model/weights/device.

        ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
preserved),
        ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
``runner.gpus=[0]``;
        cpu/mps run with no GPU list so the detector places tensors on ``device``.
        """
        from hydra import compose, initialize

        gpus = "[0]" if device == "cuda" else "[]"  # single GPU on cuda; none for cpu/mps
        with initialize(config_path="configs", version_base=None):
            cfg = compose(
                config_name="eval",
                overrides=[
                    f"dataset={sport}",
                    "model=wasb",
                    f"detector.model_path={weights}",
                    f"runner.device={device}",
                    f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to
this box
                ],
            )
        return cfg


    def _frame_id(path: str) -> int:
        """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
        stem = osp.splitext(osp.basename(path))[0]
        digits = "".join(ch for ch in stem if ch.isdigit())
        return int(digits) if digits else -1


    # ---------------------------------------------------------------------- #
    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
    # ---------------------------------------------------------------------- #
    def _cache_paths(cache_dir: str):
        return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


    def default_cache_dir(out_csv: str) -> str:
        """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
        d = osp.dirname(osp.abspath(out_csv))
        stem = osp.splitext(osp.basename(out_csv))[0]
        return osp.join(d, stem + "_wasbcache")


    def _atomic_write_text(path: str, text: str) -> None:
        """Write then rename so a crash mid-write can't leave a half-written file."""
        tmp = path + ".tmp"
        with open(tmp, "w") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
```

```
        os.replace(tmp, path)


    def _det_plain(det) -> list:
        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
        xy = det["xy"]
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


    def _dets_to_tracker(plain_list):
        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
        the tracker does vector math / np.linalg.norm on it)."""
        import numpy as np

        return [
            {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
            for p in plain_list
        ]


    def write_manifest(cache_dir: str, manifest: dict) -> None:
        _, mpath = _cache_paths(cache_dir)
        _atomic_write_text(mpath, json.dumps(manifest, indent=2))


    def read_manifest(cache_dir: str):
        _, mpath = _cache_paths(cache_dir)
        if not osp.exists(mpath):
            return None
        try:
            with open(mpath) as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            return None


    def manifest_compatible(
        manifest: dict,
        *,
        frames_dir: str,
        weights: str,
        sport: str,
        frames_in: int,
        frames_total: int,
        device: str = "cuda",
    ) -> bool:
        """A cache is reusable only if the run that produced it matches this run's inputs.

        ``device`` defaults to ``cuda`` so caches written before device-keying (no
``device``
        field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
cache
        (detections can differ numerically across devices), and vice-versa.

        ``frames_dir`` is the **already-canonical cache key** the caller stored in the
manifest
        (``run()`` passes its ``cache_key`` here, and writes the same value into the
manifest):
        ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
``video:<abspath>``
        key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
it here:
        ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
```

```
prepended,
        so a streaming cache would never match itself and every re-run would recompute
(issue #348).
        """
        if not manifest or manifest.get("version") != CACHE_VERSION:
            return False
        return (
            manifest.get("frames_dir") == frames_dir
            and manifest.get("weights") == weights
            and manifest.get("sport") == sport
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
            and manifest.get("device", "cuda") == device
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid: list = []
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break  # truncated trailing line (crash mid-flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
```

```
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)


    # ---------------------------------------------------------------------- #
    # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
    # ---------------------------------------------------------------------- #
    _STREAM_FRAME_FMT = "{:08d}.png"


    def synthetic_frame_path(index: int) -> str:
        """Stable, index-encoding frame name used by the streaming path.

        It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
        downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
        frames came from disk or from the in-memory stream. The streaming image loader
        inverts this name back to an index to fetch the decoded frame.
        """
        return _STREAM_FRAME_FMT.format(int(index))


    def build_windows(frames: list, frames_in: int):
        """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
        + checkpoint). Pure list math, identical for the disk and streaming paths.

        Returns a list of windows, each a list of ``frames_in`` consecutive paths
        (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
        caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
        ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
        """
        return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]


    class SequentialFrameStore:
        """In-memory sliding buffer over a forward-only frame reader, sized for the
        detector's sliding-window access pattern (peak O(window) frames, not O(video)).

        The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
        ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
        in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
        start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
        boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
        far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
        earliest index any not-yet-finished window can still ask for) and decode each source
        frame exactly once via the forward-only ``reader(index)``.
        """

        def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
            self._reader = reader
            self._total = int(total)
            self._window = max(1, int(window))
            self._buf: dict = {}
            # On a resumed run the detector's first needed frame is `start_index`; begin
            # pulling there so the reader can skip (seek past) the already-cached prefix
            # instead of decoding 0..start_index-1 just to throw them away.
            self._next = int(start_index)  # next index not yet pulled from the reader
            self._max_seen = int(start_index) - 1
            self._floor = int(start_index)  # lowest index we still keep (never evict below)
```

```
294
295         def get(self, index: int):
296             index = int(index)
297             if index < self._floor:
298                 raise KeyError(
299                     f"frame {index} already evicted (below floor {self._floor}); the access
"
300                     f"pattern must stay within a window of the highest frame seen"
301                 )
302             if index >= self._total:
303                 raise KeyError(f"frame {index} out of range (total {self._total})")
304             # Pull forward from the reader until the requested index is buffered.
305             while self._next <= index:
306                 self._buf[self._next] = self._reader(self._next)
307                 self._next += 1
308             if index > self._max_seen:
309                 self._max_seen = index
310             # Evict frames older than the earliest a still-running window could re-request:
311             # once we've seen index `m`, no future window starts before `m - (window-1)`.
312             new_floor = max(self._floor, self._max_seen - (self._window - 1))
313             for k in range(self._floor, new_floor):
314                 self._buf.pop(k, None)
315             self._floor = new_floor
316             return self._buf[index]
317
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326
327
328     def _bgr_to_pil_rgb(frame_bgr):
329         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
333         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
334         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
335         frame indistinguishable from the on-disk one to everything downstream.
336         """
337         import cv2
338         from PIL import Image
339
340         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
341
342
343     def _make_streamed_read_image(store, real_read_image):
344         """Build the ``read_image`` replacement used during a streaming detector pass.
345
346         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
347         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
348         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
349         without the WSL-only ``dataloaders`` package.
350         """
```

```
351
352         def _read_image(path, *args, **kwargs):
353             idx = _index_from_synthetic_path(path)
354             if idx < 0:
355                 return real_read_image(path, *args, **kwargs)
356             return _bgr_to_pil_rgb(store.get(idx))
357
358         return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
366         frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
370         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
371         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
372         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
373         ``osp.exists`` guard is sidestepped for synthetic stream paths.
374
375         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
376         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
377         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
378         reference).
379
380         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
381         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
382         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
383         original reader is restored on exit.
384         """
385         from dataloaders import dataset_loader as _dl
386
387         store = SequentialFrameStore(
388             frame_loader, total, window=window, start_index=start_index
389         )
390         real_read_image = _dl.read_image
391         # Rebind read_image in the consuming module for the duration of the detector pass;
392         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
393         _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type:
ignore[assignment]
394         try:
395             yield store
396         finally:
397             _dl.read_image = real_read_image  # type: ignore[assignment]
398
399
400     # ------------------------------------------------------------------------ #
401     # Inference
402     # ------------------------------------------------------------------------ #
403     class _PrefetchIter:
```

```
        """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).

        In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
        the cv2.imread shim live in this process and cannot be pickled to workers), so the
        CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
        measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
        A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
        release the GIL for the heavy parts, the store's forward-only access pattern is
        preserved (one consumer thread iterating sequentially), and the bounded queue caps
        memory at ``depth`` batches. A producer exception is re-raised in the consumer."""

        _END = object()

        def __init__(self, src, depth: int = 2):
            import queue
            import threading

            self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
            self._exc: "BaseException | None" = None

            def _fill() -> None:
                try:
                    for item in src:
                        self._q.put(item)
                except BaseException as e:  # noqa: BLE001 - surfaced in __next__
                    self._exc = e
                finally:
                    self._q.put(self._END)

            self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
            self._t.start()

        def __iter__(self):
            return self

        def __next__(self):
            item = self._q.get()
            if item is self._END:
                if self._exc is not None:
                    raise self._exc
                raise StopIteration
            return item


    def run(
        frames,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        num_workers: int = 4,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
        fresh: bool = False,
        frame_loader=None,
        source_key: "str | None" = None,
        device: str = "cuda",
        prefetch_batches: int = 0,
    ) -> str:
        """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
```

```
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
471         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
472           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
473           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
474           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
475           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
476           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
477           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
478           (the in-process shim + buffer can't cross worker processes).
479
480         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
481         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
482         after reboot still matches.
483         """
484         import time
485
486         import torch
487         from dataloaders import build_img_transforms, build_seq_transforms
488         from dataloaders.dataset_loader import ImageDataset
489         from torch.utils.data import DataLoader
490
491         # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
492         # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
493         # the mypy false positive where the main env resolves `trackers` to the Roboflow
package
494         # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
495         # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
496         from trackers import build_tracker  # type: ignore[attr-defined]
497         from utils import Center
498
499         from detectors import build_detector
500
501         streaming = frame_loader is not None
502
503         cfg = _build_cfg(weights, sport, device)
504         frames_in = int(cfg["model"]["frames_in"])
505         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508         if streaming:
509             # `frames` is already the ordered list of (synthetic) frame paths.
510             if not isinstance(frames, (list, tuple)):
511                 raise SystemExit(
512                     "streaming mode requires `frames` to be a list of frame paths"
513                 )
514             frames = list(frames)
515             frames_dir = source_key or "stream"
516         else:
517             frames_dir = frames
518             frames = sorted(
519                 glob.glob(osp.join(frames_dir, "*.png"))
520                 + glob.glob(osp.join(frames_dir, "*.jpg"))
521             )
```

```
        if limit:
            frames = frames[:limit]
        if len(frames) < frames_in:
            where = frames_dir if not streaming else "video stream"
            raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")

        # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
        samples = []
        for window in build_windows(frames, frames_in):
            annos = [
                {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
                for p in window
            ]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {},
            frames_dir=cache_key,
            weights=weights,
            sport=sport,
            frames_in=frames_in,
            frames_total=len(frames),
            device=device,
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(
                f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
            )
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": cache_key,
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "device": device,
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ----------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
```

```
                seq_transform_test = None
            ds = ImageDataset(
                cfg,
                remaining,
                input_wh,
                output_wh,
                transform=transform_test,
                seq_transform=seq_transform_test,
                is_train=False,
            )
            # Streaming forces single-process loading: the in-memory frame store + the
            # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
            loader_workers = 0 if streaming else num_workers
            loader = DataLoader(
                ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
            )
            detector = build_detector(cfg)

            from contextlib import ExitStack

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(
                f"detecting on {len(remaining)} remaining windows "
                f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
                f"source={'video-stream' if streaming else 'frames-dir'})..."
            )
            det_f = open(det_jsonl, "a")
            try:
                with ExitStack() as stack:
                    stack.enter_context(torch.no_grad())
                    # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
                    # in-memory decoded frame for that synthetic path; every other line of
                    # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
                    # The detector's first needed frame is `windows_done` (window i starts
at
                    # frame i), so prime the store there for a resume.
                    if streaming:
                        stack.enter_context(
                            _streaming_read_image_patch(
                                frame_loader,
                                len(frames),
                                window=frames_in,
                                start_index=windows_done,
                            )
                        )
                    # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
                    # a multi-worker DataLoader already overlaps via its worker processes).
                    batches = (
                        _PrefetchIter(loader, depth=prefetch_batches)
                        if prefetch_batches and loader_workers == 0
                        else loader
                    )
                    for bi, batch in enumerate(batches):
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [
                            list(t) for t in batch[-1]
```

```
                        ]  # [frame_in][batch] -> path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(
                                f"{done}/{windows_total} "
                                f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
                                f"elapsed {el:.0f}s | ETA {eta:.0f}s"
                            )
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2

```

```
        os.makedirs(frames_dir, exist_ok=True)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def _probe_frame_count(video_path: str) -> int:
        """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        cap.release()
        return n


    def make_video_frame_loader(video_path: str):
        """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

        ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
        long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
        detector's sliding-window order); it reads sequentially and only uses a frame-seek
to
        skip forward when a *resume* starts past the current position. Each ``cap.read()``
        returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
        yield (PNG is lossless), so the detector sees identical pixels to the disk path.

        Returns the count hint from the container so the caller can build the frame list;
        ``release`` closes the capture.
        """
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        state = {"pos": 0}  # next index cap.read() will return

        def frame_loader(index: int):
            index = int(index)
            if index < state["pos"]:
                raise RuntimeError(
                    f"streaming decode is forward-only; got index {index} after
{state['pos']}"
                )
```

```
            if index > state["pos"]:
                # Forward skip only happens when resuming past cached windows. Seek once.
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                state["pos"] = index
            ok, frame = cap.read()
            if not ok:
                raise RuntimeError(f"failed to decode frame {index} from {video_path}")
            state["pos"] += 1
            return frame

        def release():
            cap.release()

        return frame_loader, count_hint, release


    def run_video_streaming(
        video_path: str,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
        fresh: bool = False,
        device: str = "cuda",
        prefetch_batches: int = 0,
    ) -> str:
        """Fast path: decode ``video_path`` on the fly and feed frames straight to the
        detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
        dominates a long clip). Output is bit-identical to the frames-on-disk path; see
        ``run``'s docstring for why.
        """
        frame_loader, count_hint, release = make_video_frame_loader(video_path)
        try:
            if count_hint <= 0:
                raise SystemExit(
                    f"could not determine frame count for {video_path}; cannot stream "
                    f"(fall back to frame extraction with --frames_dir)"
                )
            frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
            source_key = "video:" + osp.abspath(video_path)
            return run(
                frame_list,
                weights,
                out_csv,
                sport,
                limit,
                batch_size=batch_size,
                num_workers=0,
                log_every_batches=log_every_batches,
                cache_dir=cache_dir,
                flush_every=flush_every,
                fresh=fresh,
                frame_loader=frame_loader,
                source_key=source_key,
                device=device,
                prefetch_batches=prefetch_batches,
            )
        finally:
            release()
```

```
    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument(
            "--video", help="video file; frames are extracted internally (in WSL)"
        )
        ap.add_argument(
            "--frames_out_dir", help="where to extract frames when --video is used"
        )
        ap.add_argument(
            "--stream-video",
            action="store_true",
            help="FAST PATH: decode --video on the fly and feed frames straight to "
            "the detector (no PNG extraction). Output-equivalent to the disk path.",
        )
        ap.add_argument("--weights", required=True)
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument(
            "--device",
            default="cuda",
            choices=["cuda", "mps", "cpu"],
            help="torch device for the detector (default cuda = the WSL parity path)",
        )
        ap.add_argument(
            "--limit", type=int, default=0, help="cap #frames (for quick tests)"
        )
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument(
            "--prefetch-batches",
            type=int,
            default=0,
            help="streaming mode: overlap CPU batch assembly with GPU inference via a "
            "bounded producer thread of this queue depth (0 = off, the parity default)",
        )
        ap.add_argument(
            "--log-every-batches", type=int, default=50, help="progress log cadence"
        )
        ap.add_argument(
            "--cache-dir",
            default=None,
            help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
        )
        ap.add_argument(
            "--flush-every",
            type=int,
            default=1000,
            help="fsync the detector cache every N windows (bounds reboot loss)",
        )
        ap.add_argument(
            "--fresh", action="store_true", help="ignore any existing cache and start over"
        )
        args = ap.parse_args()

        if args.stream_video:
            if not args.video:
                raise SystemExit("--stream-video requires --video")
            run_video_streaming(
                args.video,
                args.weights,
                args.out,
                args.sport,
                args.limit,
                batch_size=args.batch_size,
```

```
                log_every_batches=args.log_every_batches,
                cache_dir=args.cache_dir,
                flush_every=args.flush_every,
                fresh=args.fresh,
                device=args.device,
                prefetch_batches=args.prefetch_batches,
            )
            return

        frames_dir = args.frames_dir
        if args.video:
            frames_dir = extract_frames(
                args.video, args.frames_out_dir or (args.video + "_frames")
            )
        if not frames_dir:
            raise SystemExit("provide --frames_dir or --video")
        run(
            frames_dir,
            args.weights,
            args.out,
            args.sport,
            args.limit,
            batch_size=args.batch_size,
            num_workers=args.num_workers,
            log_every_batches=args.log_every_batches,
            cache_dir=args.cache_dir,
            flush_every=args.flush_every,
            fresh=args.fresh,
            device=args.device,
        )


    if __name__ == "__main__":
        main()

```

11. user (2026-07-06T17:10:44.981Z)

```
    {
        "sport": "badminton",
        "ai_provider": "gemini",
        "ai_model": "gemini-flash-latest",
        "validation": {
            "min_resolution_width": 1280,
            "min_resolution_height": 720,
            "min_fps": 29.9,
            "min_blur_threshold": 20.0,
            "max_scene_cuts_per_minute": 0.5,
            "relevance": {
                "court_green_lower": [
                    35,
                    40,
                    40
                ],
                "court_green_upper": [
                    85,
                    255,
                    255
                ],
                "court_pixels_min_ratio": 0.05
            }
        },
        "indexing": {
            "default_segmenter": "gemini",
            "use_gpu": true,
```

```
            "motion_threshold": 0.08,
            "min_segment_duration": 3.0,
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "detector_model": "fasterrcnn_resnet50_fpn",
            "detector_score_threshold": 0.5,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_prefetch_workers": 2,
            "min_action_window_duration": 2.0,
            "log_yolo_candidates": true,
            "store_yolo_candidates": true,
            "ai_handoff_padding": 2.0,
            "ai_max_workers": 1,
            "tracknet": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/TrackNetV4",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "TrackNetV4",
                "weights_path": "",
                "queue_length": 5,
                "stage_in_wsl": true,
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            },
            "wasb": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/WASB-SBDT",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "wasb",
                "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
                "sport": "badminton",
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            }
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1,
            "watermark": {
                "enabled": true,
                "text": "Generated by Khelsutra.guru",
                "position": "bottom-right",
                "opacity": 0.85,
                "logo_path": "",
                "font_path": ""
            }
        }
    }
```

12. assistant (2026-07-06T17:10:47.040Z)

Now let me search for the model configuration that specifies the network input resolution and any trajectory/proxy resolution definitions:

13. user (2026-07-06T17:10:47.375Z)

```
No files found
```

14. user (2026-07-06T17:10:47.712Z)

```
backend\pipeline\segmenters\wasb_hybrid.py
```

15. user (2026-07-06T17:10:48.690Z)

```
backend\reporting\spec.yaml
```

16. assistant (2026-07-06T17:10:50.279Z)

Let me read wasb_hybrid.py and search for references to resolution and proxy:

17. user (2026-07-06T17:10:50.508Z)

```
1       """WASB-based hybrid play segmenter.
2
3       Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4       WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5       badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7       For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8       candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9       min_window_duration) are what get tuned against golden rally labels.
10
11      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13      WASB runner and identity.
14      """
15
16      from typing import Any, Dict, Tuple
17
18      from backend.config.models import IndexingConfig
19      from backend.pipeline.detectors.base import DetectorRunner
20      from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
21      from backend.pipeline.segmenters.base import segmenter_registry
22      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
23
24
25      class WasbHybridSegmenter(TrajectoryHybridSegmenter):
26          family = "wasb"
27          display_label = "WASB"
28
29          @property
30          def name(self) -> str:
31              return "wasb_hybrid"
32
33          @property
34          def description(self) -> str:
35              return (
36                  "Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
37                  "weights) for precise candidate rallies + AI provider for semantic
boundaries."
38              )
39
40          def _build_runner(
41              self, idx_cfg: IndexingConfig
42          ) -> Tuple[DetectorRunner, Dict[str, Any]]:
43              cfg = WasbConfig.from_indexing_cfg(idx_cfg)
44              return WasbRunner(cfg), {"weights_path": cfg.weights_path}
```

```
45
46          def _build_native_runner(
47              self, idx_cfg: IndexingConfig
48          ) -> Tuple[DetectorRunner, Dict[str, Any]]:
49              # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
50              # seam (imported only when detector_impl='native' is actually selected).
51              from backend.pipeline.detectors.native_wasb_runner import (
52                  NativeWasbConfig,
53                  NativeWasbRunner,
54              )
55
56              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
57              return NativeWasbRunner(cfg), {
58                  "weights_path": cfg.weights_path,
59                  "detector_impl": "native",
60                  "device": cfg.device,
61              }
62
63
64      segmenter_registry.register(WasbHybridSegmenter)
65
```

18. user (2026-07-06T17:10:52.324Z)

```
Found 6 files
backend\ui\assets\index-5O3c-FCB.js
backend\worker\__main__.py
backend\pipeline\detectors\wasb_infer.py
docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
docs\data_pipeline\VIDEO_RECORDING_GUIDELINES.md
docs\archives\MONETIZATION_AUDIT.md
```

19. assistant (2026-07-06T17:10:54.312Z)

Let me search more specifically for the network resolution and proxy resolution:

20. user (2026-07-06T17:10:55.315Z)

```
No matches found
```

21. user (2026-07-06T17:10:55.885Z)

```
<persisted-output>
Output too large (31.8KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-
Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tool-
results\toolu_01YXqt46chMUb5JNdNMZ3KWJ.txt

Preview (first 2KB):
README.md:178:    It treats frame caches, staged/proxy videos, and handoff/temp dirs as
**regenerable** (purgeable) and always keeps trajectory CSVs, the SQLite DB, and final reels.
Add `--include-wasbcache` to also drop detector resume caches. `--summary` prints a one-line
nudge (used by the optional SessionStart hook).
docs\ALPHA_LAUNCH_REVIEW_2026-07-04.md:111:**OFF is only for identity-token smoke/e2e tests**
(`deploy/cloudrun/README.md:80-81`; `gen_service_yaml.py:18-20,124-131`). Why it's dangerous:
the service carries `allUsers` run.invoker so Cloudflare's proxy can reach it ? with edge-auth
ON the app-level Cf-Access JWT check is what keeps the direct `*.run.app` URL at 401
(`?a1-cuj_2026-07-04.md:94-96`); with edge-auth OFF that last gate is gone.
docs\ALPHA_LAUNCH_REVIEW_2026-07-04.md:163:[Omitted long matching line]
docs\ALPHA_LAUNCH_READINESS.md:189:[Omitted long matching line]
deploy\gcp\README.md:219:# FIRST REAL inference on the held-out Video 1 (GX010135_proxy.mp4, no
labels):
deploy\gcp\README.md:221:  --video-uri gcs://khelsutra-rally-corpus/videos/GX010135_proxy.mp4 \
deploy\gcp\README.md:238:  `gdrive://Sharing/Annotation Videos/Golden Label Videos Request -
```

June 15/[Done] Video 4 - Badminton/GX020094_proxy.mp4`
docs\CONTROL_PLANE_DURABLE_REBUILD.md:83:| D10 | Any quality-affecting flip (#483 proxy) ships a **Launch Report** gated on the golden set | standing convention [[launch-report-convention]] | honest measured results only; most-conservative-passing setting wins |
docs\CONTROL_PLANE_DURABLE_REBUILD.md:92: inference. Expected effect: detection ~22 min ? ~5 min at a 30 fps proxy.
docs\CONTROL_PLANE_DURABLE_REBUILD.md:189:threshold (from res·fps·duration vs the known ~25 fps L4 throughput), GPU-transcode a proxy
docs\CONTROL_PLANE_DURABLE_REBUILD.md:191:on the **proxy**, map rally timestamps back, cut the reel from the **ORIGINAL**. User-visible copy
...
</persisted-output>

22. assistant (2026-07-06T17:10:58.432Z)

Good?there's a "30 fps proxy" and proxy resolution mentioned. Let me search for the specific warning text about tuned proxy resolution:

23. user (2026-07-06T17:11:22.667Z)

<persisted-output>
Output too large (357.7KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tool-results\b6plx1df1.txt

Preview (first 2KB):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/ui/assets/index-5O3c-FCB.js:In order to be iterable, non-array objects must have a [Symbol.iterator]() method.`)}function Fk(l,i){if(l){if(typeof l=="string")return Hm(l,i);var r=Object.prototype.toString.call(l).slice(8,-1);if(r==="Object"&&l.constructor&&(r=l.constructor.name),r==="Map"||r==="Set")return Array.from(l);if(r==="Arguments"||/^(?:Ui|I)nt(?:8|16|32)(?:Clamped)?Array$/.test(r))return Hm(l,i)}}function Hm(l,i){(i==null||i>l.length)&&(i=l.length);for(var r=0,o=new Array(i);r<i;r++)o[r]=l[r];return o}function Yk(l,i){if(!(typeof Symbol>"u"||!(Symbol.iterator in Object(l)))){var r=[],o=!0,c=!1,d=void 0;try{for(var m=l[Symbol.iterator](),h;!(o=(h=m.next()).done)&&(r.push(h.value),!(i&&r.length===i));o=!0);}catch(g){c=!0,d=g}finally{try{!o&&m.return!=null&&m.return()}finally{if(c)throw d}}return r}}function $k(l){if(Array.isArray(l))return l}function Ur(l){"@babel/helpers - typeof";return typeof Symbol=="function"&&typeof Symbol.iterator=="symbol"?Ur=function(r){return typeof r}:Ur=function(r){return r&&typeof Symbol=="function"&&r.constructor===Symbol&&r!==Symbol.prototype?"symbol":typeof r},Ur(l)}function Bm(l,i){var r=Object.keys(l);if(Object.getOwnPropertySymbols){var o=Object.getOwnPropertySymbols(l);i&&(o=o.filter(function(c){return Object.getOwnPropertyDescriptor(l,c).enumerable})),r.push.apply(r,o)}return r}function zm(l){for(var i=1;i<arguments.length;i++){var r=arguments[i]!=null?arguments[i]:{};i%2?Bm(Object(r),!0).forEach(function(o){Xk(l,o,r[o])}):Object.getOwnPropertyDescriptors?Object.defineProperties(l,Object.getOwnPropertyDescriptors(r)):Bm(Object(r)).forEach(function(o){Object.defineProperty(l,o,Object.getOwnPropertyDescriptor(r,o))})}return l}function Xk(l,i,r){return i in l?Object.defineProperty(l,i,{value:r,enumerable:!0,configurable:!0,writable:!0}):l[i]=r,l}function Qk(l,i){if(!(l instanceof i))throw new TypeError("Cannot call a class as a function")}function Ik(
...
</persisted-output>

24. assistant (2026-07-06T17:11:25.307Z)

Let me search more specifically in the code:

25. user (2026-07-06T17:11:25.922Z)

backend\worker__main__.py:294:   "detections, or the input resolution differs from the tuned proxy resolution. "

26. assistant (2026-07-06T17:11:27.738Z)

Perfect! Let me read that file around that line:

27. user (2026-07-06T17:11:27.979Z)

```
280             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
281             if not segments:
282                 detector_impl = (
283                     os.environ.get("RALLY_DETECTOR_IMPL")
284                     or getattr(config.indexing, "detector_impl", None)
285                     or "auto"
286                 )
287                 logger.warning(
288                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
289                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
290                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
291                     "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
292                     "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
293                     "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
294                     "detections, or the input resolution differs from the tuned proxy
resolution. "
295                     "Re-run with -v for the native command + frame counts.",
296                     video_id,
297                     segmenter_actual,
298                     detector_impl,
299                 )
300         finally:
301             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
302             emit_indexer_report(
303                 db=db,
304                 video_id=video_id,
305                 video_path=local_video,
306                 config=config,
307                 val_results=val_results,
308                 val_context=val_context,
309                 segmenter_requested=args.segmenter,
310                 segmenter_actual=segmenter_actual,
311                 was_reingest=False,
312                 segmentation_ran=segmentation_ran,
313             )
314
315         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
316         out_local = os.path.join(scratch, "outputs", video_id)
317         os.makedirs(out_local, exist_ok=True)
318         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
319         _write_report_json(
320             video_id,
321             segments,
322             status,
323             os.path.join(out_local, "report.json"),
324             video_uri=str(getattr(args, "video_uri", "") or ""),
325         )
326
327         out_prefix = args.out_uri.rstrip("/")
328         pushed = []
329         for fn in sorted(os.listdir(out_local)):
```

28. assistant (2026-07-06T17:11:29.917Z)

The warning just mentions "tuned proxy resolution" but doesn't define it. Let me search for where that resolution value is actually defined:

29. user (2026-07-06T17:11:31.168Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/worker/__main__.py:
"detections, or the input resolution differs from the tuned proxy resolution. "
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/tests/test_annotation_proxy.py:    assert ap.needs_proxy("x.mp4", max_height=720) is
True   # 1080 > 720 ? worth a proxy
```

30. assistant (2026-07-06T17:11:33.446Z)

Let me look at the CONTROL_PLANE doc that was mentioned in the grep output earlier and search for fps/stride/frame skip:

31. user (2026-07-06T17:11:34.415Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/fusion_features.py:    True (count everyone) when it is None.
Unusable frame dims (w/h <= 0) make the foot
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        frame_skip: int = 3,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:            frame_skip: Process every Nth
frame (1 = every frame). Higher values
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        # Effective sampling rate after
frame-skipping
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        effective_fps = fps /
max(frame_skip, 1)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:            # Skip frames for performance
? at 30fps, every 3rd frame still
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:            if frame_idx % max(frame_skip,
1) != 0:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        self, video_path: str,
start_frame: int, end_frame: int, frame_skip: int = 3
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        step = max(frame_skip, 1)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        frame_skip: int = 3,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py:        step = max(frame_skip, 1)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/stub_runner.py:        well above the default 0.02 velocity
threshold ? so every rally frame is "active"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/tracknet_runner.py:        temporal midpoint until every
piece is ? ``max_win_frames`` (makes the cap a true upper
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/wasb_infer.py:trajectory is cheap and *stateful* (it must see
every frame in order, so it can't
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/wasb_infer.py:    [x, y, score, scale] candidates
(accumulated across every window the frame is in,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors/wasb_infer.py:    detector, WITHOUT extracting every frame to
a PNG first (the disk round-trip that
```

32. assistant (2026-07-06T17:11:35.903Z)

Good! Now let me check if WASB itself does any frame skipping or if it processes all frames:

33. user (2026-07-06T17:11:36.994Z)

```
16:The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
21:  * The detector pass writes its per-window raw outputs incrementally to
22:    ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
24:    most ``--flush-every`` windows of GPU work instead of the whole pass.
25:  * On restart we replay the cached windows, skip the DataLoader ahead to the first
26:    un-cached window, and continue. Once every window is cached, the detector pass
32:to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33:weights, sport, window size, or frame count change.
151:    frames_in: int,
174:        and manifest.get("frames_in") == frames_in
193:    Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194:    [x, y, score, scale] candidates (accumulated across every window the frame is in,
195:    in window order ? identical to a non-resumed run).
220:def _append_windows(fh, objs) -> None:
221:    """Append window records as JSONL and durably flush (bounded loss on crash)."""
241:# Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
257:def build_windows(frames: list, frames_in: int):
258:    """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
261:    Returns a list of windows, each a list of ``frames_in`` consecutive paths
262:    (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
263:    caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
266:    return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
271:    detector's sliding-window access pattern (peak O(window) frames, not O(video)).
274:    ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275:    in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276:    start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277:    boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278:    far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279:    earliest index any not-yet-finished window can still ask for) and decode each source
283:    def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
286:        self._window = max(1, int(window))
300:                f"pattern must stay within a window of the highest frame seen"
310:        # Evict frames older than the earliest a still-running window could re-request:
311:        # once we've seen index `m`, no future window starts before `m - (window-1)`.
312:        new_floor = max(self._floor, self._max_seen - (self._window - 1))
363:    frame_loader, total: int, window: int = 1, start_index: int = 0
381:    in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``); on a
382:    resume we start at ``start_index`` so the decoder seeks past already-cached windows. The
388:        frame_loader, total, window=window, start_index=start_index
406:    In streaming mode the DataLoader must stay single-process (the in-memory frame store +
```

34. assistant (2026-07-06T17:11:38.553Z)

Let me look at the key section where frames_in is read from the config:

35. user (2026-07-06T17:11:39.402Z)

```
500
501         streaming = frame_loader is not None
502
503         cfg = _build_cfg(weights, sport, device)
504         frames_in = int(cfg["model"]["frames_in"])
505         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508         if streaming:
509             # `frames` is already the ordered list of (synthetic) frame paths.
510             if not isinstance(frames, (list, tuple)):
511                 raise SystemExit(
512                     "streaming mode requires `frames` to be a list of frame paths"
```

```
513                 )
514             frames = list(frames)
515             frames_dir = source_key or "stream"
516         else:
517             frames_dir = frames
518             frames = sorted(
519                 glob.glob(osp.join(frames_dir, "*.png"))
```

36. assistant (2026-07-06T17:11:41.798Z)

So `frames_in`, `inp_width`, and `inp_height` come from the WASB model config. Let me check if there are command-line args that allow frame skipping:

37. user (2026-07-06T17:11:42.770Z)

```
37:        --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-every
1000] [--fresh]
40:import argparse
836:    ap = argparse.ArgumentParser()
837:    ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
838:    ap.add_argument(
841:    ap.add_argument(
844:    ap.add_argument(
850:    ap.add_argument("--weights", required=True)
851:    ap.add_argument("--out", required=True)
852:    ap.add_argument("--sport", default="badminton")
853:    ap.add_argument(
859:    ap.add_argument(
860:        "--limit", type=int, default=0, help="cap #frames (for quick tests)"
862:    ap.add_argument("--batch-size", type=int, default=8)
863:    ap.add_argument("--num-workers", type=int, default=4)
864:    ap.add_argument(
871:    ap.add_argument(
874:    ap.add_argument(
879:    ap.add_argument(
885:    ap.add_argument(
```

38. assistant (2026-07-06T17:11:44.394Z)

Now let me check the actual argument handling to see what --limit does:

39. user (2026-07-06T17:11:44.617Z)

```
520                 + glob.glob(osp.join(frames_dir, "*.jpg"))
521             )
522         if limit:
523             frames = frames[:limit]
524         if len(frames) < frames_in:
525             where = frames_dir if not streaming else "video stream"
526             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
529         samples = []
```

40. assistant (2026-07-06T17:11:46.287Z)

So `--limit` just caps the total frames?it doesn't do regular frame skipping. Now let me search for where prefetch_batches and batch_size are actually used in the GPU inference:

41. user (2026-07-06T17:11:46.638Z)

```
596             # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
```

```
597             loader_workers = 0 if streaming else num_workers
598             loader = DataLoader(
599                 ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600             )
601             detector = build_detector(cfg)
602 
603             from contextlib import ExitStack
604 
605             t0 = time.time()
606             done = windows_done
607             win_idx = windows_done
608             log_every = max(1, log_every_batches)
609             flush_n = max(1, flush_every)
610             pending = []
611             logger.info(
612                 f"detecting on {len(remaining)} remaining windows "
613                 f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614                 f"source={'video-stream' if streaming else 'frames-dir'})..."
615             )
616             det_f = open(det_jsonl, "a")
617             try:
618                 with ExitStack() as stack:
619                     stack.enter_context(torch.no_grad())
620                     # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621                     # in-memory decoded frame for that synthetic path; every other line of
622                     # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
623                     # The detector's first needed frame is `windows_done` (window i starts
at
624                     # frame i), so prime the store there for a resume.
625                     if streaming:
626                         stack.enter_context(
627                             _streaming_read_image_patch(
628                                 frame_loader,
629                                 len(frames),
630                                 window=frames_in,
631                                 start_index=windows_done,
632                             )
633                         )
634                     # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635                     # a multi-worker DataLoader already overlaps via its worker processes).
636                     batches = (
637                         _PrefetchIter(loader, depth=prefetch_batches)
638                         if prefetch_batches and loader_workers == 0
639                         else loader
640                     )
641                     for bi, batch in enumerate(batches):
642                         imgs, _hms, trans = batch[0], batch[1], batch[2]
643                         img_paths = [
644                             list(t) for t in batch[-1]
645                         ]  # [frame_in][batch] -> path
646                         batch_results, _ = detector.run_tensor(imgs, trans)
647                         nb = imgs.shape[0]
648                         for ib in range(nb):
649                             frames_payload = []
650                             for ie in sorted(batch_results[ib].keys()):
651                                 p = img_paths[ie][ib]
652                                 fid = _frame_id(p)
653                                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                                 frames_payload.append([fid, plain])
655                                 det_by_fid[fid].extend(plain)
```

42. assistant (2026-07-06T17:11:48.194Z)

Good! Now let me look at what the config.json serves as default for wasb batch_size and prefetch_batches:

43. user (2026-07-06T17:11:48.398Z)


```
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-flash-latest",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [
                35,
                40,
                40
            ],
            "court_green_upper": [
                85,
                255,
                255
            ],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "gemini",
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
        "detector_model": "fasterrcnn_resnet50_fpn",
        "detector_score_threshold": 0.5,
        "yolo_velocity_threshold": 0.15,
        "yolo_frame_skip": 3,
        "yolo_tracker": "bytetrack.yaml",
        "yolo_prefetch_workers": 2,
        "min_action_window_duration": 2.0,
        "log_yolo_candidates": true,
        "store_yolo_candidates": true,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 1,
        "tracknet": {
            "wsl_distro": "Ubuntu",
            "repo_dir": "~/models/TrackNetV4",
            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
            "conda_env": "TrackNetV4",
            "weights_path": "",
            "queue_length": 5,
            "stage_in_wsl": true,
            "wsl_stage_dir": "~/clips",
            "timeout_sec": 1800,
            "velocity_threshold": 0.02
        },
        "wasb": {
            "wsl_distro": "Ubuntu",
            "repo_dir": "~/models/WASB-SBDT",
            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
            "conda_env": "wasb",
```

```
61                  "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62                  "sport": "badminton",
63                  "wsl_stage_dir": "~/clips",
64                  "timeout_sec": 1800,
65                  "velocity_threshold": 0.02
66              }
67          },
68          "output": {
69              "default_highlights_name": "highlights.mp4",
70              "db_name": "sports_indexer.db"
71          },
72          "stitching": {
73              "begin_padding": 0.5,
74              "end_padding": 0.5,
75              "use_cache": false,
76              "min_shot_count": 1,
77              "watermark": {
78                  "enabled": true,
79                  "text": "Generated by Khelsutra.guru",
80                  "position": "bottom-right",
81                  "opacity": 0.85,
82                  "logo_path": "",
83                  "font_path": ""
84              }
85          }
86      }
```

44. assistant (2026-07-06T17:11:50.017Z)

I notice the wasb section doesn't have batch_size or prefetch_batches. Let me check the IndexingConfig model to see default values:

45. user (2026-07-06T17:12:00.495Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/config/models.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/windowing.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/native_wasb_runner.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/wasb_runner.py
```

46. assistant (2026-07-06T17:12:01.743Z)

Let me read the config models:

47. user (2026-07-06T17:12:02.984Z)

```
118:    batch_size: Optional[int] = None
121:    prefetch_batches: Optional[int] = None
123:    @field_validator("batch_size")
125:    def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
130:            raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got {v})")
133:    @field_validator("prefetch_batches")
135:    def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
141:                f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
```

48. assistant (2026-07-06T17:12:04.391Z)

Good! Now let me read more of that file to see the defaults:

49. user (2026-07-06T17:12:04.663Z)

```
100         # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
```

```
not warns).
        # 0 = no extra floor (the headroom guard still applies).
        min_free_gb: float = 0.0
        # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
        # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
        # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
        # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
        # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
        # Tri-state:
        #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
        #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
        #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
        stream_video: Optional[bool] = None
        # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
        # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
        # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
        # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
        # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
        batch_size: Optional[int] = None
        # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
        # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
        prefetch_batches: Optional[int] = None

        @field_validator("batch_size")
        @classmethod
        def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
            # A DataLoader batch must be >=1; reject at config time so a typo can't reach
            # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
            # when truthy, so 0 stays "off/parity-default").
            if v is not None and v < 1:
                raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
            return v

        @field_validator("prefetch_batches")
        @classmethod
        def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
            # 0 = off (the default). Reject negatives: the runner passes any truthy value as
            # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
            # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
            if v is not None and v < 0:
                raise ValueError(
                    f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
                )
            return v


    class FusionConfig(BaseModel):
        """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).

        Every default reproduces control behaviour: the fusion segmenter persists the
        shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
```

```
is
151         true ? the person-cue features, but it does NOT gate the served handoff unless a
152         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
153         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
154         supplied ? off-court persons (spectators / adjacent courts) are excluded.
155         """
156
157         # Which trajectory substrate seeds the windows (shuttle stays primary).
158         substrate: Literal["wasb", "tracknet"] = "wasb"
159         # Windowing velocity threshold for the fusion family (the engine reads
160         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
161         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
162         velocity_threshold: float = 0.02
163         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
164         # enabled ? so the default path never imports torch / touches the GPU.
165         cue_flags: dict[str, bool] = Field(default_factory=dict)
166         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
167         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
168         min_crossings: int = Field(default=0, ge=0)
169         # Served Gate B: when the person cue is on, drop windows with median in-court
players
170         # < this. 0 = OFF.
171         min_players: int = Field(default=0, ge=0)
172         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
173         court_polygon: Optional[list[list[float]]] = None
174         # Net line for the person two-sided-motion split (person features only ? the shuttle
175         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
176         net_axis: Literal["x", "y"] = "y"
177         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
178
179
```

50. assistant (2026-07-06T17:12:06.387Z)

Now let me check the wasb_infer.py defaults for batch_size:

51. user (2026-07-06T17:12:07.152Z)

```
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
453         limit: int = 0,
454         batch_size: int = 8,
455         num_workers: int = 4,
456         log_every_batches: int = 50,
457         cache_dir: "str | None" = None,
458         flush_every: int = 1000,
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",
463         prefetch_batches: int = 0,
464     ) -> str:
465         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
```

52. assistant (2026-07-06T17:12:08.848Z)

Perfect! Now let me search for any documentation on what the network resolution is. Let me check the CONTROL_PLANE document that was mentioned:

53. user (2026-07-06T17:12:10.172Z)

| D10 | Any quality-affecting flip (#483 proxy) ships a **Launch Report** gated on the golden set | standing convention [[launch-report-convention]] | honest measured results only; most-conservative-passing setting wins |
  inference. Expected effect: detection ~22 min ? ~5 min at a 30 fps proxy.
threshold (from res·fps·duration vs the known ~25 fps L4 throughput), GPU-transcode a proxy on the **proxy**, map rally timestamps back, cut the reel from the **ORIGINAL**. User-visible copy
explains it. **Gate (D10):** golden-set (n=15) F1/tolF1 at the proxy settings vs baseline; ship
| Proxy detection regresses quality (#483) | worse rallies for testers | D10 golden gate + Launch Report before default-ON; most-conservative-passing setting |

54. assistant (2026-07-06T17:12:11.720Z)

The document mentions "30 fps proxy" but doesn't define the spatial resolution. Let me search for any actual test or config files that define the model input size:

55. user (2026-07-06T17:12:13.000Z)

Exit code 123

56. assistant (2026-07-06T17:12:14.557Z)

Let me try a different approach - search for where the values are actually read from WASB config:

57. user (2026-07-06T17:12:15.739Z)

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: frames_in: int,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: and manifest.get("frames_in") == frames_in
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py:def build_windows(frames: list, frames_in: int):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: Returns a list of windows, each a list of ``frames_in`` consecutive paths
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``); on a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: frames_in = int(cfg["model"]["frames_in"])
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: if len(frames) < frames_in:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py: raise SystemExit(f"need >= {frames_in}

```
frames, found {len(frames)} in {where}")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/wasb_infer.py:    # Sliding windows of `frames_in`
consecutive frames (the unit of GPU work + checkpoint).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/wasb_infer.py:    for window in build_windows(frames,
frames_in):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/wasb_infer.py:        frames_in=frames_in,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/wasb_infer.py:        "frames_in": frames_in,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/backend/pipeline/detectors/wasb_infer.py:                            window=frames_in,
```

58. assistant (2026-07-06T17:12:18.942Z)

The resolution and frames_in come from `cfg["model"]` which is loaded from the WASB repo's config via Hydra. Let me now check if there's any actual downsampling or proxy logic in the pipeline:

59. user (2026-07-06T17:12:20.040Z)

(Bash completed with no output)

60. assistant (2026-07-06T17:12:21.468Z)

Let me check the trajectory_hybrid file for any resolution-related logic:

61. user (2026-07-06T17:12:22.503Z)

```
121:        return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
```

62. assistant (2026-07-06T17:12:24.938Z)

Now let me look at the ImageDataset transform to understand if there's any frame resizing in the model's inference path:

63. user (2026-07-06T17:12:25.935Z)

```
4:own building blocks (ImageDataset transform + detector + online tracker) to run on
264:    ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
274:    ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
365:    """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory decoded
368:    WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
470:      globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via ``cv2.imread``.
475:      ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
487:    from dataloaders import build_img_transforms, build_seq_transforms
488:    from dataloaders.dataset_loader import ImageDataset
581:        _, transform_test = build_img_transforms(cfg)
583:            _, seq_transform_test = build_seq_transforms(cfg)
585:            seq_transform_test = None
586:        ds = ImageDataset(
591:            transform=transform_test,
592:            seq_transform=seq_transform_test,
620:                # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)` to
the
```

64. assistant (2026-07-06T17:12:27.664Z)

The ImageDataset is from the WASB repo (imported from `dataloaders`). The transform is built from WASB's config via `build_img_transforms(cfg)`. Let me search for any window size information in the code:

65. user (2026-07-06T17:12:28.736Z)

```
16:The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
33:weights, sport, window size, or frame count change.
174:        and manifest.get("frames_in") == frames_in
271:    detector's sliding-window access pattern (peak O(window) frames, not O(video)).
381:    in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``); on a
504:    frames_in = int(cfg["model"]["frames_in"])
550:        frames_in=frames_in,
613:            f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
748:    detector's sliding-window order); it reads sequentially and only uses a frame-seek to
```

66. assistant (2026-07-06T17:12:30.534Z)

Good. Now let me check if there's a scaling/resizing operation between the source video and the network input. Let me look at the ImageDataset usage more carefully:

67. user (2026-07-06T17:12:30.804Z)

```
580         if remaining:
581             _, transform_test = build_img_transforms(cfg)
582             try:
583                 _, seq_transform_test = build_seq_transforms(cfg)
584             except Exception:
585                 seq_transform_test = None
586             ds = ImageDataset(
587                 cfg,
588                 remaining,
589                 input_wh,
590                 output_wh,
591                 transform=transform_test,
592                 seq_transform=seq_transform_test,
593                 is_train=False,
594             )
595             # Streaming forces single-process loading: the in-memory frame store + the
596             # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597             loader_workers = 0 if streaming else num_workers
598             loader = DataLoader(
599                 ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
```

68. assistant (2026-07-06T17:12:33.093Z)

So `input_wh` is passed to ImageDataset. This is `(int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))` from line 505. The ImageDataset transform will resize to this `input_wh`. Let me check if there are any unit tests or examples that show what this resolution actually is:

69. user (2026-07-06T17:12:34.634Z)

```
(Bash completed with no output)
```

70. assistant (2026-07-06T17:12:36.156Z)

Let me check the actual command that native_wasb_runner constructs to see what it passes:

71. user (2026-07-06T17:12:36.530Z)

```
258             weights = os.path.expanduser(self.cfg.weights_path)
259             argv = [
260                 self.cfg.python_bin,
261                 "wasb_infer.py",
262                 "--video",
263                 os.path.abspath(str(video_win_path)),
```

```
264             ]
265             frames_dir: Optional[str] = None
266             if self.cfg.stream_video:
267                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
268                 argv.append("--stream-video")
269             else:
270                 scratch = self.cfg.scratch_dir or output_dir
271                 frames_dir = os.path.join(scratch, f"{key}_frames")
272                 argv += ["--frames_out_dir", frames_dir]
273             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275             device = self._effective_device()
276             argv += [
277                 "--weights",
278                 weights,
279                 "--sport",
280                 self.cfg.sport,
281                 "--device",
282                 device,
283                 "--out",
284                 out_csv,
285             ]
286             # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287             # byte-parity guarantee) is unchanged when the knobs are unset.
288             if self.cfg.batch_size:
289                 argv += ["--batch-size", str(self.cfg.batch_size)]
290             if self.cfg.prefetch_batches:
291                 argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
```

72. assistant (2026-07-06T17:12:38.277Z)

Perfect! Now let me look at how batching and prefetch affect throughput by examining the _PrefetchIter class more closely:

73. user (2026-07-06T17:12:38.492Z)

```
403     class _PrefetchIter:
404         """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).
405
406         In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
407         the cv2.imread shim live in this process and cannot be pickled to workers), so the
408         CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
409         measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410         A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
411         release the GIL for the heavy parts, the store's forward-only access pattern is
412         preserved (one consumer thread iterating sequentially), and the bounded queue caps
413         memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
414
415         _END = object()
416
417         def __init__(self, src, depth: int = 2):
418             import queue
419             import threading
420
421             self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
422             self._exc: "BaseException | None" = None
423
```

```
424             def _fill() -> None:
425                 try:
426                     for item in src:
427                         self._q.put(item)
428                 except BaseException as e:  # noqa: BLE001 - surfaced in __next__
429                     self._exc = e
430                 finally:
431                     self._q.put(self._END)
432 
433             self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
434             self._t.start()
435 
436         def __iter__(self):
437             return self
438 
439         def __next__(self):
440             item = self._q.get()
441             if item is self._END:
442                 if self._exc is not None:
443                     raise self._exc
444                 raise StopIteration
445             return item
446 
447 
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
```

74. assistant (2026-07-06T17:12:40.485Z)

Now let me summarize the findings by checking all the key points I need to answer. Let me create a final confirmation search for each point:

Point 1: Per-frame network resolution

75. user (2026-07-06T17:12:41.341Z)

```
    streaming = frame_loader is not None

    cfg = _build_cfg(weights, sport, device)
    frames_in = int(cfg["model"]["frames_in"])
    input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
    output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))

    if streaming:
        # `frames` is already the ordered list of (synthetic) frame paths.
        if not isinstance(frames, (list, tuple)):
```

76. assistant (2026-07-06T17:12:45.038Z)

Now let me check for mentions about whether source resolution matters:

77. user (2026-07-06T17:12:46.333Z)

(Bash completed with no output)

78. assistant (2026-07-06T17:12:47.827Z)

Let me check the trajectory parsing to understand what information is captured:

79. user (2026-07-06T17:12:48.131Z)

```
228     def _atomic_write_trajectory(out_csv: str, rows) -> None:
229         os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
230         tmp = out_csv + ".tmp"
231         with open(tmp, "w", newline="") as f:
232             w = csv.writer(f)
233             w.writerow(["Frame", "Visibility", "X", "Y"])
234             w.writerows(rows)
235             f.flush()
236             os.fsync(f.fileno())
237         os.replace(tmp, out_csv)
238
239
240     # ------------------------------------------------------------------------ #
241     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242     # ------------------------------------------------------------------------ #
```

80. assistant (2026-07-06T17:13:19.205Z)

Good. Now let me compile my findings into a comprehensive report:

FINDINGS REPORT: WASB Shuttle-Detector Inference Performance Characteristics

1. **Per-Frame Network Input Resolution (FIXED RESOLUTION)**

File: `backend/pipeline/detectors/wasb_infer.py`, line 505

```python
input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
```

Finding: The WASB network resizes **every input frame to a FIXED network resolution** loaded from the WASB model config via Hydra:
- This resolution is **not defined in the indexer codebase** ? it comes from the WASB-SBDT repo's model config (`~/models/WASB-SBDT/configs/eval.yaml` or similar under the cloned WASB repo)
- The indexer only **reads and uses this resolution**; it does NOT control it
- The `ImageDataset` from WASB's dataloaders applies `transform_test` which resizes frames to `input_wh` before forward pass

SOURCE VIDEO RESOLUTION IMPACT: The indexer code shows no special handling of source resolution (1920x1080 vs 1280x720). The resize cost is incurred ONCE per frame via the transform pipeline (`build_img_transforms(cfg)`), regardless of source dimensions. The forward-pass GPU cost is **independent of source resolution** because the network always operates on the fixed `input_wh`. Only the **CPU-side preprocessing cost** (decode + resize) scales with source resolution, but this is amortized in batching and overlapped with GPU via `_PrefetchIter`.

---

2. **Frame Selection / Temporal Sampling (ALL FRAMES PROCESSED)**

Finding: **ALL frames are processed** ? there is **NO frame-skipping knob** in the WASB detection path.

Evidence:
- `wasb_infer.py` line 504: `frames_in = int(cfg["model"]["frames_in"])` ? this is the **sliding-window size**, not a stride/skip parameter
- `build_windows()` at line 257-266 constructs **overlapping sliding windows** with stride=1:
  ```python
  def build_windows(frames: list, frames_in: int):
      return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
  ```
  This yields `len(frames) - frames_in + 1` windows ? **every frame becomes the start of a window** (except the last `frames_in - 1`).

--limit parameter (line 860): Only used for testing to cap total frames; not for regular frame skipping.

Result: A 60 fps video **processes all 60 frames per second of video**. No fps-reduction knob exists. The frame rate of output depends only on the input video's frame rate.

---

3. **Throughput Levers: batch_size and prefetch_batches**

File: `backend/pipeline/detectors/wasb_infer.py` and `backend/pipeline/detectors/native_wasb_runner.py`

batch_size (default: 8)
- **Definition** (line 454): `batch_size: int = 8` in the `run()` function signature
- **Usage** (line 599): Passed to `DataLoader(ds, batch_size=batch_size, ...)`
- **Effect:** Controls the **number of windows processed in a single GPU forward pass** (not frames; each window contains `frames_in` frames). Larger batches improve GPU utilization and throughput (measured on L4 serving run, 2026-07-03, GPU was <1% utilized at batch=8 with single-worker loading).
- **Config control** (backend/config/models.py, lines 118, 123-130): `batch_size: Optional[int] = None` ? defaults to wasb_infer's hardcoded 8; can be overridden via `indexing.wasb.batch_size` in config.json (currently not set, so uses default)

prefetch_batches (default: 0, OFF)
- **Definition** (line 463): `prefetch_batches: int = 0` in the `run()` function signature
- **Usage** (lines 636-640): Creates a bounded producer thread that overlaps **CPU-side batch assembly (decode + PIL + ToTensor)** with **GPU inference**:
  ```python
  batches = (
      _PrefetchIter(loader, depth=prefetch_batches)
      if prefetch_batches and loader_workers == 0
      else loader
  )
  ```
- **_PrefetchIter** (lines 403-445): A bounded queue (max depth = `prefetch_batches`) that decouples CPU-side frame loading from GPU inference via a background thread
- **Effect:** In streaming mode (single-process loader, `num_workers=0`), this was measured to leave GPU <1% utilized; a prefetch depth of 2+ overlaps CPU work with GPU, improving throughput
- **Config control** (backend/config/models.py, lines 121, 133-143): `prefetch_batches: Optional[int] = None` ? defaults to 0 (OFF); can be enabled via `indexing.wasb.prefetch_batches` in config.json (currently not set, so disabled)
- **Current serving defaults** (config.json, lines 56-66): The `indexing.wasb` section does **NOT specify batch_size or prefetch_batches**, so both remain at wasb_infer.py defaults (batch=8, prefetch=0).

How raising them changes throughput:
- **Raising batch_size:** More windows per GPU forward pass ? higher throughput (windows/sec), but higher peak GPU memory
- **Enabling prefetch_batches (e.g., depth=2):** Overlaps CPU decode/transform with GPU inference, hiding CPU latency ? higher sustained throughput, especially in streaming mode where there's no multi-worker DataLoader parallelism

---

4. **"Tuned Proxy Resolution" Definition**

File: `backend/worker/__main__.py`, line 294

```python
"detections, or the input resolution differs from the tuned proxy resolution. "
```

Finding: The phrase "tuned proxy resolution" appears **only in a warning message** and is **not defined anywhere in the codebase**. This is a **forward-looking reference** to a planned

proxy video feature (downsampling videos before inference).

Related context (from CONTROL_PLANE_DURABLE_REBUILD.md):
- Line 92: "Expected effect: detection ~22 min ? ~5 min at a 30 fps proxy" ? a 30 fps downsampled video
- Lines 189-191: "GPU-transcode a proxy ... on the **proxy**, map rally timestamps back, cut the reel from the **ORIGINAL**"

Current status: This feature is documented as planned but **not yet implemented**. There is no proxy resolution hardcoded or enforced in the detection path.

---

5. **Existing Spatial-Downscale or Fps-Reduction Steps**

Finding: **No spatial downsampling or fps-reduction steps exist in the detection code path** and **none are enabled by default**.

Evidence:
- No `--downsample`, `--scale`, or `--reduce-fps` arguments in wasb_infer.py
- No proxy-video generation in the segmentation pipeline
- No stride/skip knob in the sliding-window construction (stride=1, all frames)
- The only resolution manipulation is the **fixed network input resize** done by ImageDataset's transform

Potential future optimization (mentioned in warning but not implemented):
- The CONTROL_PLANE document discusses a future "proxy video" path: transcode the input to a lower fps/resolution for detection, then map results back to the original timing
- This would **NOT be in wasb_infer.py** but in the segmenter's preparation phase (trajectory_hybrid.process_video)

SUMMARY TABLE

Aspect	Value / Behavior	File:Line	Current Config Default
Network input resolution	Fixed (from WASB model config: `cfg["model"]["inp_width/height"]`)	wasb_infer.py:505	External (WASB repo config)
Source resolution impact	Only affects CPU decode/resize cost; GPU forward cost is independent	wasb_infer.py:505-506	N/A
Frame selection	ALL frames processed; sliding windows with stride=1	wasb_infer.py:257-266	N/A
FPS reduction knob	NONE	?	N/A
batch_size (GPU batching)	Batches windows for throughput	wasb_infer.py:599	8 (hardcoded default)
prefetch_batches (CPU-GPU overlap)	Thread-based queue for streaming mode	wasb_infer.py:637	0 (disabled by default)
Config file knobs	`indexing.wasb.batch_size`, `indexing.wasb.prefetch_batches`	backend/config/models.py:118,121	Both `None` (use hardcoded defaults)
Spatial downsampling	NOT implemented	?	N/A
FPS downsampling	NOT implemented	?	N/A
Proxy resolution (tuned)	Documented as planned, not yet implemented	worker/__main__.py:294	N/A

PROOF THAT DOWNSAMPLING WOULD ACCELERATE PROCESSING

1. **CPU decode + resize cost** scales linearly with source resolution ? downsampling reduces this
2. **GPU forward cost** scales with network input size (fixed `inp_width × inp_height`) ? independent of source resolution
3. **Memory footprint** during frame buffering scales with source resolution ? downsampling

reduces this
4. **Batch assembly in streaming mode** (decode + PIL + ToTensor) is currently GPU-underutilizing and bottleneck ? downsampling + optional prefetch_batches would improve throughput measurably

To prove: Measure wall-clock time for a 1920x1080 video vs a downsampled 1280x720 proxy with the same frame count (or same fps but fewer frames). The difference would isolate the CPU-side cost that downsampling eliminates.